



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 13 de febrero
de 2026

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	4
Apéndice B Especificación de Requisitos	5
B.1. Introducción	5
B.2. Objetivos generales	5
B.3. Catálogo de requisitos	5
B.4. Especificación de requisitos	5
B.5. Actores del sistema	6
Apéndice C Especificación de diseño	11
C.1. Introducción	11
C.2. Diseño de datos	11
C.3. Diseño arquitectónico	13
C.4. Diseño procedimental	14
Apéndice D Documentación técnica de programación	19
D.1. Introducción	19
D.2. Estructura de directorios	19

D.3. Manual del programador	19
D.4. Compilación, instalación y ejecución del proyecto	19
D.5. Pruebas del sistema	19
Apéndice E Documentación de usuario	21
E.1. Introducción	21
E.2. Requisitos de usuarios	21
E.3. Instalación	21
E.4. Manual del usuario	21
Apéndice F Anexo de sostenibilización curricular	23
F.1. Introducción	23
Bibliografía	25

Índice de figuras

B.1. Diagrama de Casos de Uso	6
C.1. Mockup de la web	16
C.2. Diagrama de Flujo de la Aplicación	17
C.3. Diagrama de Secuencia de la aplicación	17

Índice de tablas

C.1. Comparativa de ventajas e inconvenientes entre Bootstrap y Tailwind CSS	15
--	----

Apéndice A

Plan de Proyecto Software

A.1. Introducción

La planificación de un proyecto software constituye un proceso de gestión organizado que permite coordinar actividades, recursos y plazos con el fin de garantizar una ejecución controlada y la consecución de los objetivos definidos. En el contexto de este TFG, donde se desarrolla una aplicación web para la inferencia y visualización de sendas de herbívoros a partir de imágenes aéreas, disponer de un plan de proyecto resulta especialmente relevante para estructurar el trabajo y registrar el avance de forma verificable.

Este apartado presenta, en primer lugar, la planificación temporal seguida durante el desarrollo, desglosando el proyecto en tareas e hitos y estableciendo su evolución cronológica. Para ello, se adopta un enfoque iterativo e incremental inspirado en metodologías ágiles, particularmente *Scrum*, organizando el trabajo en ciclos cortos llamados *Sprints* con los que se busca conseguir una correcta revisión del progreso y una posible adaptación ante incidencias o cambios de alcance, manteniendo una visión continua del conjunto de tareas pendientes y completadas mediante el denominado *Product Backlog*, que funciona como un registro de las labores a llevar a cabo.

Finalmente, el plan se completa con un estudio de viabilidad que analiza el proyecto desde una perspectiva económica y legal. En esta parte se consideran los costes asociados al desarrollo y despliegue, así como las implicaciones derivadas del uso de herramientas, librerías y recursos de terceros, prestando especial atención a licencias software y condiciones de

uso, con el objetivo de asegurar que la solución propuesta sea sostenible, reutilizable y coherente con un escenario de explotación real.

A.2. Planificación temporal

Sprint 7

El *Sprint 7* abarcó el periodo comprendido entre el 21 de enero y el 10 de febrero. Se trató de un intervalo especialmente amplio, en el que se concentraron cambios relevantes tanto en la memoria como en la implementación, en consonancia con el cierre de mi periodo de prácticas y el consecuente incremento del tiempo dedicado al TFG. En conjunto, planifiqué una dedicación total de 39,5 horas para este *sprint*, aunque finalmente empleé 54 horas.

Los objetivos definidos para este *sprint* fueron los siguientes:

1. Revisar y reformular el apartado de objetivos del proyecto, con el propósito de mejorar su claridad y consistencia.
2. Refinar explicaciones del Capítulo 3, *Conceptos teóricos*, corrigiendo definiciones puntuales.
3. Reestructurar el Capítulo 6, *Trabajos relacionados*, incorporando mayor apoyo visual y completando explicaciones incompletas.
4. Redactar los bloques conceptuales pendientes correspondientes a:
 - Métricas de evaluación en segmentación semántica,
 - Tipos de *encoder*.
5. Integrar en la aplicación web la lógica del algoritmo de inferencia proporcionado.
6. Adaptar la batería de tests a los cambios introducidos.
7. Generar un fichero `requirements.txt` con las dependencias de Python del proyecto.
8. Elaborar el diagrama de casos de uso y documentar los casos asociados.

En cuanto a la revisión de los objetivos, la organicé en dos días. El primero lo dediqué a identificar las descripciones que resultaban menos

precisas y a analizar memorias de años anteriores para ajustar el estilo al esperado, mientras que el segundo se centró en la reescritura y el refinamiento final del texto.

Paralelamente, corregí definiciones concretas del Capítulo 3, en particular las relativas a *Vision Transformer* y a los mecanismos residuales. Asimismo, incorporé un texto introductorio al Capítulo 6, revisé algunas redacciones y ajusté *captions* de las figuras. Finalmente, añadí una tabla comparativa entre trabajos relacionados, cuya construcción y ajuste visual realicé en dos iteraciones para asegurar su correcta visualización en el documento.

Más adelante, completé los conceptos pendientes, correspondientes a métricas de evaluación y tipos de *encoder*. La elaboración se distribuyó en cinco días: dos destinados a métricas de evaluación, primero para redactar el contenido y después para consolidar bibliografía, fórmulas y recursos visuales, y tres destinados a *encoders*, donde fue necesario realizar una revisión bibliográfica más amplia antes de cerrar la redacción.

El hito central de este *sprint* fue la integración en la aplicación web de la lógica de inferencia del *notebook* proporcionado. Tras analizar el flujo completo, abordé una primera aproximación mediante serialización con *pickle*, la cual no conseguí llegar a implementar del todo correctamente. Por ello, opté por una solución más robusta basada en convertir los modelos por *fold* a módulos *TorchScript* autocontenidos, ya normalizados y directamente integrables en la aplicación. Posteriormente, adapté la lógica del sistema para ejecutar esta inferencia de forma funcional, cerrando esta parte el 1 de febrero.

Como consecuencia de los cambios introducidos, actualicé la batería de tests para alinearla con la nueva integración. Adicionalmente, por recomendación de uno de los tutores, generé el fichero `requirements.txt` con las versiones de las dependencias utilizadas hasta el momento, al disponer ya de la información necesaria.

Por último, finalicé otra tarea arrastrada de *sprints* anteriores: la creación del diagrama de casos de uso y la documentación asociada. Para ello, me apoyé en los *mockups* y en el flujo de uso previsto, lo que facilitó mantener una estructura coherente y cerrada.

Cabe destacar que, antes de la reunión que dio inicio al siguiente *sprint*, se realizó una reunión intermedia de seguimiento. Sin embargo, al considerar que todavía quedaban varios frentes abiertos y que el avance no era suficientemente cerrado, se decidió prolongar el *sprint* una semana adicional para consolidar entregables y reducir la deuda técnica acumulada.

A.3. Estudio de viabilidad

Viabilidad económica

Viabilidad legal

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

B.2. Objetivos generales

B.3. Catálogo de requisitos

B.4. Especificación de requisitos

En esta sección se presenta la especificación funcional de la aplicación mediante casos de uso, describiendo de forma estructurada las interacciones que el usuario puede realizar a través de la interfaz *frontend*. Dado que la aplicación se consume desde el cliente web, el diagrama sintetiza las funcionalidades visibles para cada tipo de actor, mientras que el *backend* se considera un soporte imprescindible para ejecutar la inferencia, gestionar el estado de las imágenes y servir los resultados, sin modelarse explícitamente como actor independiente.

Diagrama de Casos de Uso

En la figura [B.1](#) se recoge el diagrama general de casos de uso y, a continuación, se detallan las descripciones individuales de cada caso representado. Estas tablas se han derivado a partir del flujo de uso previsto y de los *mocups* de la interfaz, con el objetivo de concretar precondiciones, secuencias principales, relaciones entre funcionalidades y el nivel de importancia de cada operación dentro del *pipeline* de la aplicación.

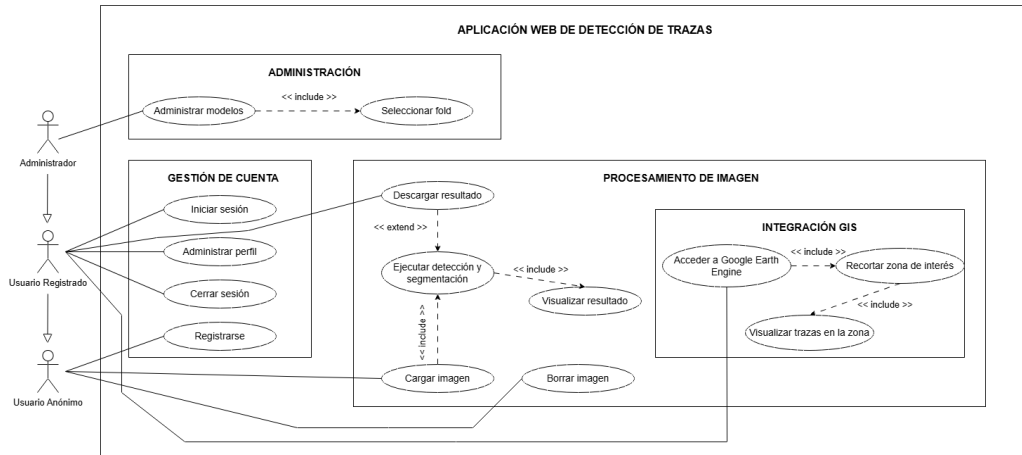


Figura B.1: Diagrama de casos de uso de la aplicación

B.5. Actores del sistema

En esta sección se describen los distintos actores que interactúan con la aplicación web de detección de trazas. Los actores representan los perfiles de usuario y roles funcionales que acceden al sistema, diferenciándose en función de sus permisos y capacidades dentro de la aplicación. Esta distinción permite modelar de forma clara las responsabilidades y el alcance de cada tipo de usuario.

Usuario Anónimo

El **Usuario Anónimo** representa a cualquier persona que accede a la aplicación sin haber iniciado sesión ni disponer de una cuenta registrada. Este actor tiene acceso a las funcionalidades básicas del sistema, orientadas a la demostración y uso general de la herramienta de detección.

Concretamente, el Usuario Anónimo puede:

- Cargar una imagen para su análisis.
- Ejecutar el proceso de detección y segmentación de trazas.
- Visualizar el resultado obtenido.
- Borrar la imagen cargada y reiniciar el flujo de procesamiento.
- Registrarse en el sistema.

Este perfil permite el uso inmediato de la aplicación sin barreras de acceso, manteniendo restringidas aquellas funcionalidades que requieren persistencia de datos o integración con servicios externos.

Usuario Registrado

El **Usuario Registrado** es un agente que dispone de una cuenta en el sistema y ha iniciado sesión correctamente. Este actor hereda todas las capacidades del Usuario Anónimo y, adicionalmente, puede acceder a funcionalidades avanzadas que requieren autenticación.

Además de las acciones disponibles para el Usuario Anónimo, el Usuario Registrado puede:

- Iniciar y cerrar sesión en el sistema.
- Administrar su perfil de usuario.
- Descargar los resultados obtenidos tras el proceso de detección y segmentación.
- Acceder al módulo de integración GIS mediante Google Earth Engine.
- Definir una zona de interés y visualizar trazas dentro de dicha zona.

Este actor representa el perfil principal de uso de la aplicación, proporcionando acceso a funcionalidades de mayor valor añadido y a la integración con servicios externos.

Administrador

El **Administrador** es un usuario registrado con privilegios adicionales orientados a la gestión y mantenimiento del sistema. Este actor hereda todas las capacidades del Usuario Registrado y dispone de permisos específicos para la configuración interna de la aplicación.

Las funciones exclusivas del Administrador incluyen:

- Administrar los modelos de detección utilizados por el sistema.
- Seleccionar el modelo o *fold* activo que se empleará en el proceso de detección y segmentación.

La existencia de este rol permite separar claramente las tareas de uso del sistema de aquellas relacionadas con su configuración y control, facilitando la mantenibilidad y escalabilidad de la aplicación.

Explicación de los casos de uso

CU-01 Cargar imagen

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite al usuario cargar una imagen aérea que será utilizada como entrada para el proceso de detección y segmentación de trazas.
Precondición	El usuario ha accedido a la interfaz de procesamiento.
Flujo principal	1) El usuario selecciona una imagen desde su dispositivo. 2) El sistema valida el formato y tamaño del archivo. 3) La imagen queda cargada y disponible para su procesamiento.
Postcondición	Imagen cargada correctamente en el sistema.
Relaciones	Incluido por CU-02 Ejecutar detección y segmentación.
Importancia	Alta

CU-02 Ejecutar detección y segmentación

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Ejecuta el modelo de detección y segmentación de trazas sobre la imagen cargada.
Precondición	Existe una imagen cargada (CU-01).
Flujo principal	1) El usuario inicia el proceso de análisis. 2) El sistema aplica el modelo de detección configurado. 3) Se generan los resultados de segmentación.
Postcondición	Resultado de detección disponible para su visualización.
Relaciones	Incluye CU-01 Cargar imagen. Es incluido por CU-03 Visualizar resultado. Es extendido por CU-04 Descargar resultado.
Importancia	Alta

CU-03 Visualizar resultado

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite visualizar el resultado de la detección y segmentación superpuesto sobre la imagen original.
Precondición	El proceso de detección se ha ejecutado correctamente (CU-02).
Flujo principal	1) El sistema renderiza la imagen original. 2) El sistema superpone el resultado de segmentación. 3) El usuario inspecciona visualmente el resultado.
Postcondición	Resultado visualizado en pantalla.
Relaciones	Incluye CU-02 Ejecutar detección y segmentación.
Importancia	Alta

CU-04 Descargar resultado

Actores	Usuario registrado, Administrador
Descripción	Permite descargar el resultado de la detección y segmentación para su uso externo.
Precondición	El resultado ha sido generado y visualizado (CU-03).
Flujo principal	1) El usuario solicita la descarga del resultado. 2) El sistema genera el archivo correspondiente. 3) El archivo es descargado por el usuario.
Postcondición	Resultado almacenado localmente por el usuario.
Relaciones	Extiende CU-02 Ejecutar detección y segmentación.
Importancia	Media-Alta

CU-05 Borrar imagen

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite eliminar la imagen cargada y reiniciar el flujo de procesamiento.
Precondición	Existe una imagen cargada o un resultado generado.
Flujo principal	1) El usuario solicita borrar la imagen. 2) El sistema elimina la imagen y resultados asociados. 3) El sistema vuelve al estado inicial.
Postcondición	Sistema preparado para una nueva ejecución.
Importancia	Media

CU-06 Acceder a Google Earth Engine

Actores	Usuario registrado, Administrador
Descripción	Permite acceder al módulo de integración con Google Earth Engine para trabajar sobre una zona geográfica.
Precondición	El usuario ha iniciado sesión.
Flujo principal	1) El usuario accede al módulo GIS. 2) El sistema establece conexión con Google Earth Engine. 3) Se habilitan las funcionalidades de recorte y visualización.
Postcondición	Sesión GIS activa.
Relaciones	Incluye CU-06.1 Recortar zona de interés y CU-06.2 Visualizar trazas en la zona.
Importancia	Media-Alta

CU-06.1 Recortar zona de interés

Actores	Usuario registrado, Administrador
Descripción	Permite definir una zona geográfica de interés mediante un recorte sobre el visor.
Precondición	Acceso al módulo GEE (CU-06).
Flujo principal	1) El usuario define el área de recorte. 2) El sistema valida la geometría. 3) La zona queda establecida como área activa.
Postcondición	Zona de interés definida.
Importancia	Media

CU-06.2 Visualizar trazas en la zona

Actores	Usuario registrado, Administrador
Descripción	Permite visualizar las trazas detectadas dentro de la zona de interés seleccionada.
Precondición	Zona de interés definida (CU-06.1).
Flujo principal	1) El sistema consulta las trazas dentro de la zona. 2) El sistema renderiza las trazas sobre el visor. 3) El usuario inspecciona el resultado.
Postcondición	Trazas visualizadas en la zona seleccionada.
Importancia	Media-Alta

Apéndice C

Especificación de diseño

C.1. Introducción

C.2. Diseño de datos

Comparativa entre Bootstrap y Tailwind CSS

En el diseño e implementación de la interfaz web del proyecto resulta necesario seleccionar un *framework* de estilos que aporte productividad sin comprometer la mantenibilidad ni la calidad del código. Entre las alternativas más consolidadas se encuentran Bootstrap, orientado a componentes predefinidos, y Tailwind CSS, basado en un paradigma de utilidades. A continuación se presenta una comparativa crítica entre ambas opciones, justificando la elección final de Tailwind CSS para el desarrollo de esta página.

Bootstrap propone un conjunto amplio de componentes listos para usar, tales como barras de navegación, tarjetas o sistemas de rejilla, acompañados de un diseño visual relativamente homogéneo. Esta aproximación permite, en un primer momento, acelerar el desarrollo, ya que se pueden componer interfaces funcionales mediante la combinación de bloques estándar. Además, Bootstrap ofrece una documentación madura y una comunidad extensa, lo que facilita la resolución de dudas y la búsqueda de ejemplos; sin embargo, esta misma estandarización introduce ciertas limitaciones. Por un lado, la personalización profunda del diseño suele requerir sobrescribir reglas de estilo, lo que incrementa la complejidad de las hojas CSS y genera deuda técnica con facilidad. Por otro lado, las interfaces tienden a parecerse entre

sí, de forma que la identidad visual del proyecto puede quedar diluida si no se realiza un esfuerzo considerable de adaptación.

En contraste, Tailwind CSS se basa en clases utilitarias de bajo nivel que describen propiedades concretas, tales como márgenes, tipografía o colores, aplicadas directamente sobre el marcado HTML. Esta filosofía permite construir componentes a medida a partir de combinaciones de utilidades, sin depender de un conjunto rígido de componentes predefinidos. Como consecuencia, el diseño resultante se ajusta mejor a los requisitos específicos del proyecto y favorece la creación de un sistema de diseño propio, más alineado con la identidad visual deseada. Además, Tailwind CSS integra mecanismos de purga de clases no utilizadas, lo que reduce de manera significativa el tamaño del fichero CSS final y mejora el rendimiento en producción.

No obstante, Tailwind CSS presenta también ciertos inconvenientes. Al inicio, la curva de aprendizaje puede parecer más pronunciada, ya que obliga a pensar en términos de propiedades de bajo nivel y no tanto en componentes cerrados. Asimismo, la presencia de múltiples clases en cada elemento puede dar lugar a un marcado aparentemente más denso, pero, cuando se interioriza el conjunto de utilidades y se adoptan buenas prácticas, como la extracción de componentes reutilizables y el uso de configuración centralizada, el resultado es un código más predecible y coherente, con menos necesidad de sobrescrituras complejas.

Desde una perspectiva de la ingeniería del software, la elección entre ambos enfoques afecta de forma directa a la mantenibilidad y escalabilidad del proyecto. Bootstrap ofrece rapidez inicial, pero puede derivar en hojas de estilo difíciles de gestionar cuando se requiere un alto grado de personalización. En cambio, Tailwind CSS promueve una relación más estrecha entre el diseño y el código, ya que el sistema de diseño se expresa mediante utilidades consistentes que se reutilizan en toda la aplicación. Esto reduce la duplicidad de estilos, minimiza el riesgo de efectos colaterales al modificar clases existentes y facilita la evolución del producto a largo plazo.

Finalmente, he optado por Tailwind CSS debido a su capacidad para generar interfaces más ligeras, a su enfoque modular y a su mejor integración con patrones modernos de desarrollo basado en componentes. Gracias a su sistema de utilidades, ha sido posible definir un diseño adaptado a las necesidades concretas del proyecto, mantener un control fino sobre el *bundle* de estilos y reducir la deuda técnica asociada a la personalización del *frontend*. En definitiva, Tailwind CSS proporciona un equilibrio más adecuado entre

flexibilidad, rendimiento y mantenibilidad, lo que lo convierte en la opción más idónea para este desarrollo.

C.3. Diseño arquitectónico

Arquitectura general de la aplicación

La Figura C.2 representa la arquitectura general de la aplicación y las interacciones entre sus distintos componentes, diferenciando claramente el lado cliente y el lado servidor.

El usuario interactúa con la aplicación a través del navegador web, que actúa como intermediario entre la interfaz gráfica y el backend. En el cliente se distinguen dos responsabilidades bien separadas. Por un lado, el HTML renderizado mediante Jinja se encarga de mostrar la estructura de la interfaz, incluyendo botones, imágenes y modales informativos. Por otro lado, la lógica implementada en JavaScript, apoyada en un elemento *canvas*, gestiona la descarga de los resultados en formato JSON y su representación gráfica, sin delegar operaciones de modificación visual al servidor.

El servidor, implementado con Flask, centraliza la lógica de negocio y expone los distintos endpoints a través de un *Blueprint* dedicado a la gestión de trazas. Para mantener el estado de la aplicación entre peticiones HTTP, se utiliza una *session cookie* que almacena información relativa a la imagen activa y a la disponibilidad de resultados calculados. Asimismo, el sistema emplea almacenamiento en disco, dentro del directorio *instance*, tanto para persistir las imágenes subidas por el usuario como para guardar los resultados intermedios en formato JSON.

Este diseño garantiza una separación clara de responsabilidades, evitando que el cliente modifique directamente recursos en el servidor y relegando el procesamiento visual al navegador, lo que mejora la escalabilidad y simplifica la arquitectura.

Flujo temporal de interacción entre componentes

La Figura C.3 describe de forma cronológica el flujo de interacción entre el usuario, el navegador, el servidor Flask y el sistema de almacenamiento, permitiendo comprender el comportamiento dinámico de la aplicación.

El proceso se inicia cuando el usuario selecciona una imagen desde la interfaz. El navegador envía dicha imagen al servidor mediante una petición

POST upload, tras lo cual Flask la persiste en disco y actualiza el estado de la sesión. A continuación, el servidor responde con una redirección que provoca la recarga de la página principal, mostrando la imagen cargada.

Posteriormente, cuando el usuario solicita el cálculo de trazas, el servidor ejecuta la lógica correspondiente, genera los resultados en formato JSON y los almacena en el sistema de ficheros, actualizando de nuevo la sesión para reflejar el nuevo estado. La respuesta se materializa en una recarga de la interfaz que incluye un modal de confirmación.

Una vez cerrado el modal, el navegador solicita explícitamente los resultados mediante una petición *GET traces*. El servidor lee el fichero JSON previamente generado y lo devuelve al cliente, donde el código JavaScript se encarga de dibujar las trazas sobre el *canvas* y reflejar visualmente el estado de la aplicación.

Finalmente, si el usuario decide eliminar la imagen, se envía una petición *POST delete*. El servidor elimina tanto la imagen como los resultados asociados, limpia la información de la sesión y devuelve la aplicación a su estado inicial mediante una nueva recarga de la página.

C.4. Diseño procedimental

Aspecto	Bootstrap	Tailwind CSS
Enfoque	Conjunto de componentes predefinidos y estilos de alto nivel	Clases utilitarias de bajo nivel que permiten construir componentes a medida
Curva de aprendizaje	Accesible al inicio gracias a los componentes estándar y a los ejemplos completos	Inicialmente más exigente, requiere pensar en términos de propiedades y sistemas de diseño
Productividad inicial	Elevada en prototipos y desarrollos con requisitos visuales genéricos	Crece con el tiempo, especialmente cuando se definen patrones reutilizables y componentes propios
Personalización del diseño	Personalización profunda costosa, necesidad frecuente de sobrescribir estilos y romper el diseño base	Alta capacidad de personalización mediante combinaciones de utilidades y configuración centralizada
Mantenibilidad	Riesgo de hojas CSS voluminosas y difíciles de refactorizar en proyectos muy adaptados	Estilos más controlados, menor necesidad de sobrescrituras y reducción de la deuda técnica
Rendimiento y tamaño del CSS	Tamaño mayor en el fichero final cuando se utilizan muchos componentes sin optimización específica	Tamaño significativamente reducido gracias a la purga de utilidades no utilizadas en el producto final
Identidad visual	Interfaces con aspecto reconocible y similar a otros proyectos basados en el mismo <i>framework</i>	Mayor capacidad para definir una identidad visual propia y diferenciada
Comunidad y ecosistema	Comunidad muy amplia, gran cantidad de plantillas, ejemplos y recursos formativos	Comunidad en fuerte crecimiento, ecosistema consolidado en proyectos modernos basados en componentes

Tabla C.1: Comparativa de ventajas e inconvenientes entre Bootstrap y Tailwind CSS

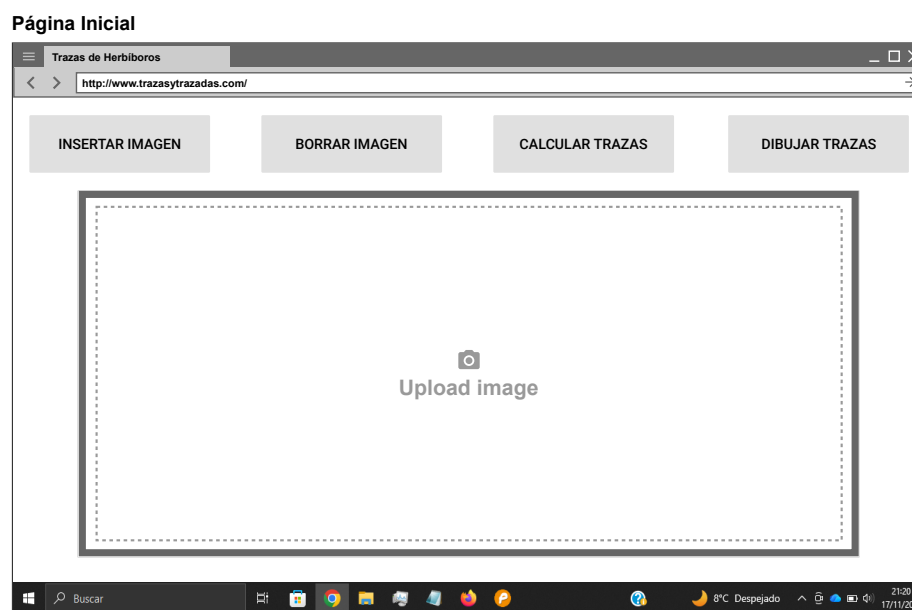


Figura C.1: Mockup de la web

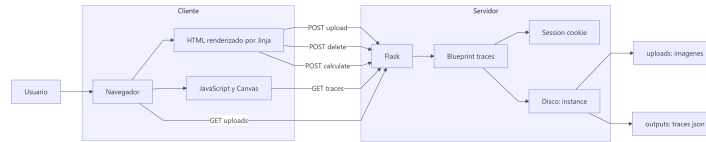


Figura C.2:

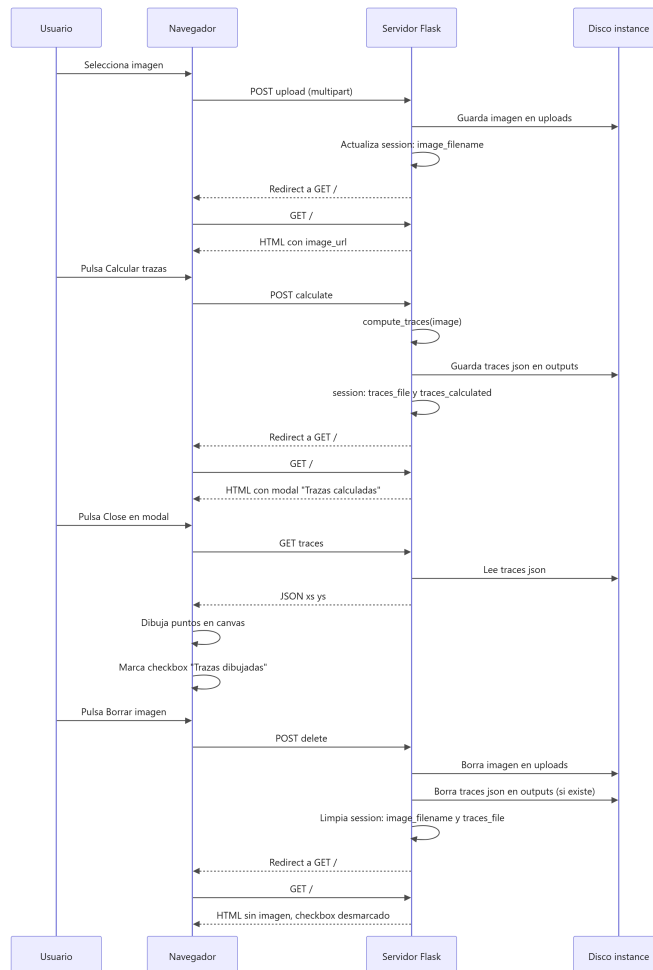


Figura C.3:

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía
